

CENG 232

Logic Design

Spring 2025-2026

Lab Assignment 2

Due date: Friday, April 10, 2026, 23:59

1 Introduction

This laboratory aims to familiarize you with some of the most important IC components, such as multiplexers and decoders. You are expected to design your circuit using the CENG232 Logisim tool with the given gates below and implement it via Verilog HDL.

2 IC Pool

- 74HC08 (AND)
- 74HC32 (OR)
- 74HC04 (NOT, Inverter)
- 74HC02 (NOR)
- 74HC00 (NAND)
- 74HC153 (Multiplexer)
- 74HC86 (XOR)
- 74HC138 (Decoder)

3 Lab Work

Modern-day CPUs and microcontroller units (MCUs) can feature specialized hardware that performs specific tasks, such as running machine learning algorithms, cryptography, signal processing, and graphics rendering, in addition to arithmetic logic units (ALUs). As this hardware is designed specifically for these tasks, mathematically heavy operations demanded by them can be performed faster and more energy efficiently, compared to standard ALUs.

In this lab, you are expected to design and build a specialized hardware module that can perform several standard and non-standard arithmetic/logic operations. The module is controlled via a 7-bit input. The three most significant bits (A6, A5, A4) determine which operation should be performed, and the remaining bits (A3, A2, A1, A0) act as inputs to the arithmetic/logic operations. Depending on the operation considered, the module should activate one or more outputs.

The results of the arithmetic/logic operations are represented with the following 1-bit outputs:

- **Z**: indicates whether an arithmetic operation result is 0.
- **E**: indicates whether the inputs are equal to each other for the logical comparison operations

- **G**: indicates whether the first input is greater than the second one for the logical comparison operations
- **L**: indicates whether the first input is less than the second one for the logical comparison operations
- **I**: indicates whether an invalid arithmetic/logic operation is requested
- **O3**: The most significant bit of the result of an arithmetic operation
- **O2**: The second most significant bit of the result of an arithmetic operation
- **O1**: The third most significant bit of the result of an arithmetic operation
- **O0**: The least significant bit of the result of an arithmetic operation

Table 1 lists all functionality that should be implemented by the module. The module performs the following eight arithmetic/logic operations: 2-bit addition, 2-bit unsigned comparison, 4-bit increment, bitwise XNOR, 4-bit range check, 2-bit multiplication, 2-bit 2's complement signed comparison, and 4-bit binary to gray code conversion.

Selection Input			Operation	Affected Outputs
A6	A5	A4		
0	0	0	2-bit Addition	Z, O3, O2, O1, O0
0	0	1	2-bit Comparison	E, G, L
0	1	0	4-bit Increment	Z, I, O3, O2, O1, O0
0	1	1	Bitwise XNOR	Z, O1, O0
1	0	0	4-bit Range Check	Z, O3, O2, O1, O0
1	0	1	2-bit Multiplication	Z, O3, O2, O1, O0
1	1	0	2-bit 2's Complement Signed Comparison	E, G, L
1	1	1	4-bit Binary to Gray Code Conversion	O3, O2, O1, O0

Table 1: The operation selection table for the module. The "Affected Outputs" indicate which output group can be active for a particular operation

In the following subsections, you can find detailed information about the arithmetic/logic operations considered by the module.

3.1 Arithmetic/Logic Operations

In Operations 1, 2, 4, 6, and 7, the 4-bit input is split into two 2-bit operands: **X = A3A2** (first operand) and **Y = A1A0** (second operand). In Operations 3, 5, and 8, the full 4-bit input **A3A2A1A0** is treated as a single unsigned integer.

Operation 1: 2-bit Addition (A6A5A4 = 000)

Computes the unsigned sum of $X = A3A2$ and $Y = A1A0$. The result is at most 3 bits wide ($O3 = 0$ always).

$$O2O1O0 = A3A2 + A1A0, \quad O3 = 0$$

$$Z = 1 \iff \text{result} = 0, \quad E=G=L=I = 0 \text{ (always)}$$

Example: $A3=1, A2=0, A1=1, A0=1 \Rightarrow X=2, Y=3, X+Y=5=101_2 \Rightarrow O2=1, O1=0, O0=1, O3=0, Z=0$.

Operation 2: 2-bit Comparison (A6A5A4 = 001)

Compares $X = A3A2$ and $Y = A1A0$ as unsigned integers.

$$E = 1 \iff A3A2 = A1A0, \quad G = 1 \iff A3A2 > A1A0, \quad L = 1 \iff A3A2 < A1A0$$

$$I=O3=O2=O1=O0=Z = 0 \text{ (always)}$$

Exactly one of E, G, L is 1 for any input.

Example: $A3=1, A2=1, A1=0, A0=1 \Rightarrow X=3, Y=1, X>Y \Rightarrow G=1, E=0, L=0$.

Operation 3: 4-bit Increment (A6A5A4 = 010)

Computes $A3A2A1A0 + 1$ modulo 16.

$$O3O2O1O0 = (A3A2A1A0 + 1) \bmod 16$$

$$Z = 1 \iff \text{result} = 0, \quad I = 1 \iff A3A2A1A0 = 1111 \text{ (overflow)}, \quad E=G=L = 0 \text{ (always)}$$

Note: When the input is 1111, both Z and I are 1 simultaneously (overflow, result wraps to 0).

Example: $A3=0, A2=1, A1=1, A0=0 \Rightarrow \text{input}=6, \text{result}=7=0111_2 \Rightarrow O3=0, O2=1, O1=1, O0=1, Z=0, I=0$.

Operation 4: Bitwise XNOR (A6A5A4 = 011)

Computes the bitwise XNOR of $X = A3A2$ and $Y = A1A0$.

$$O1 = A3 \odot A1, \quad O0 = A2 \odot A0, \quad O3=O2 = 0 \text{ (always)}$$

$$Z = 1 \iff O1 = 0 \text{ AND } O0 = 0, \quad E=G=L=I = 0 \text{ (always)}$$

Example: $A3=1, A2=0, A1=1, A0=1 \Rightarrow O1=1 \odot 1=0, O0=0 \odot 1=1 \Rightarrow O1=0, O0=1, Z=0$.

Operation 5: 4-bit Range Check (A6A5A4 = 100)

Classifies the input $A3A2A1A0$ into one of four ranges using a one-hot encoding.

Input Range	Values	O3 O2 O1 O0
0 – 3	0, 1, 2, 3	0 0 0 1
4 – 7	4, 5, 6, 7	0 0 1 0
8 – 11	8, 9, 10, 11	0 1 0 0
12 – 15	12, 13, 14, 15	1 0 0 0

Table 2: Range classification encoding for Operation 5.

$$Z = 1 \iff A3=A2=A1=A0=0, \quad E=G=L=I = 0 \text{ (always)}$$

Example: $A3=1, A2=0, A1=1, A0=0 \Rightarrow \text{input}=10, \text{range } 8\text{--}11 \Rightarrow O3=0, O2=1, O1=0, O0=0, Z=0$.

Operation 6: 2-bit Multiplication (A6A5A4 = 101)

Computes the multiplication of unsigned $X = A3A2$ and unsigned $Y = A1A0$.

$$O3O2O1O0 = A3A2 * A1A0$$

$$Z = 1 \iff \text{result} = 0, \quad E=G=L=I = 0 \text{ (always)}$$

Example: $A3=1, A2=1, A1=1, A0=1 \Rightarrow X=3, Y=3, X*Y=9=1001_2 \Rightarrow O3=1, O2=0, O1=0, O0=1, Z=0$.

Operation 7: 2-bit 2's Complement Signed Comparison (A6A5A4 = 110)

Compares $X = A3A2$ and $Y = A1A0$ as signed integers in 2's complement.

$$E = 1 \iff X = Y, \quad G = 1 \iff X > Y, \quad L = 1 \iff X < Y$$

$$I=O3=O2=O1=O0=Z = 0 \text{ (always)}$$

Exactly one of E, G, L is 1 for any input.

Example: $A3=1, A2=1, A1=0, A0=0 \Rightarrow X=-1, Y=0, X<Y \Rightarrow G=0, E=0, L=1$.

Operation 8: 4-bit Binary-gray Converter (A6A5A4 = 111)

Generates the following gray codes (O3O2O1O0) for a given 4-bit input (A3A2A1A0) with the following conversion table:

Binary Code				Gray Code			
A ₃	A ₂	A ₁	A ₀	O ₃	O ₂	O ₁	O ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Table 3: 4-bit binary to gray code conversion table

$$Z=E=G=L=I = 0 \text{ (always)}$$

Example: A3=1, A2=0, A1=1, A0=1 $\Rightarrow$ O3=1, O2=1, O1=1, O0=0.

Note: You are expected to implement your circuitry using only the ICs specified in the IC pool section.

4 Free Session

After your homework is announced, there will be a *free session week*. You will have 2 hours in your free session slot. At the very beginning, a couple of the new ICs (for this assignment) will be introduced. In the remaining time of the free session, you may build your solution circuit on a breadboard by using IC components and practice how to handle possible problems related to a physical circuit on a breadboard.

5 Demo Session

There will be a 2-hour-long *demo session week* following the free session week. In the demo session:

- You will take a short quiz about the logic concepts that involve the coverage of this lab.
- You will reconstruct your circuit on your breadboard.
- You will show that the circuit drawn in Logisim works as specified.
- A sample solution that can be implemented will be published before the first demo session (**you are not going to be asked to implement your whole design**).

6 Labelling Specifications

- You have to use **pins** for your inputs and outputs. Please only set **label property** of the **pin** objects; do not add a **text object**.
- Your input pins should be labelled as A6, A5, A4, A3, A2, A1, and A0.
- Your output pins should be labelled as Z, E, G, L, I, O3, O2, O1, and O0.
- Label properties are case-sensitive. Note that all labels consist of an uppercase letter followed by a number. **Please be very careful with the naming of labels.**
- If you need to feed any input with a constant value, you can use a constant gate. This gate is under the CENG232 Gates section. We will only set values for A6, A5, A4, A3, A2, A1, and A0. If you add an input instead of a constant gate in order to provide a 0 or 1 value for a logic gate in your design, during evaluation, your design may get very low scores even if it has been implemented correctly.
- You will receive a grade **penalty** unless labeling is done correctly.

7 Verilog Implementation (Additional Requirement)

In addition to the Logisim design, you are also required to implement the hardware module described previously using **Verilog HDL**.

7.1 Module Specification

You must implement a combinational Verilog module with the following interface:

```
module lab2(  
    input [6:0] A,  
    output Z,  
    output E,  
    output G,  
    output L,  
    output I,  
    output [3:0] O  
);  
    // Your implementation  
endmodule
```

7.2 Design Requirements

- Your design must be purely **combinational**.
- You can define auxiliary modules in addition to the **lab2** module in the same ".v" file (e.g., multiplexer, decoder, etc.) to simplify your module implementation.
- You can utilize the basic logic gate primitives already implemented by Verilog, such as **and**, **or**, **xor**, **nand**, etc.
- You must not use any behavioral delay statements.
- You may use continuous assignments (**assign**) or combinational **always @(*)** blocks.
- The Verilog implementation must produce the same outputs as your Logisim circuit for all possible input combinations.
- The evaluation will be done using automated test cases (black-box testing).

8 Deliverables

1. Please submit your circuit design file named `e1234567.circ` prepared in Logisim and your implementation file named `e1234567.v`, which is your preliminary work, until the specified deadline. For both files, please do not forget to replace `e1234567` with your 7-digit student ID. The evaluation of the submission will be a black-box test. **You should use the CENG version of Logisim is available on the ODTUClass course page. Circuits designed with other Logisim versions, other tools, or not named properly will not be graded!**
2. In the demo session, you will reconstruct and show that the circuit drawn in Logisim works physically. This part will be graded in the lab.

Note: Your overall Lab 2 grade is going to be calculated from the Logisim assignment score, Verilog assignment score, lab demo score, and quiz score.

9 What to Bring to the Demo Lab

- Circuit designs prepared with CENG232 Logisim (you can have them printed or downloaded on your phone/tablet, etc.).
- Data sheets of chips (you can have them printed or downloaded on your phone/tablet, etc.). www.alldatasheet.com
- Pencil and eraser, as you will have a quiz at the very beginning of demo sessions.

Note: The lab materials (gates, cables, etc.) will be distributed at the beginning of each lab session and will be collected at the end of the session. Therefore, you do not have to buy them. If you want to work out of the labs, you may still buy them. Moreover, if you would like to work with your own equipment (ICs, breadboard, cables, etc), you are free to bring them to the sessions as well.

10 Cheating Policy

All lab work should be done individually, and there is a zero-tolerance policy for cheating. Please see the course website for further information about the cheating policy.

11 References

- CENG 232 Logisim
- Discussions Page